

**SIMFORGE:
A REPRODUCIBLE CROSS-SIMULATOR
TESTING FRAMEWORK**

AGENDA



Motivation & Research Gap



Research Questions & Expected Contributions



Background & Simulator Scope



SIMFORGE Framework & Workflow



Metrics & Evaluation Plan



Risks & Timeline



MOTIVATION & RESEARCH GAP



WHY THIS WORK MATTERS

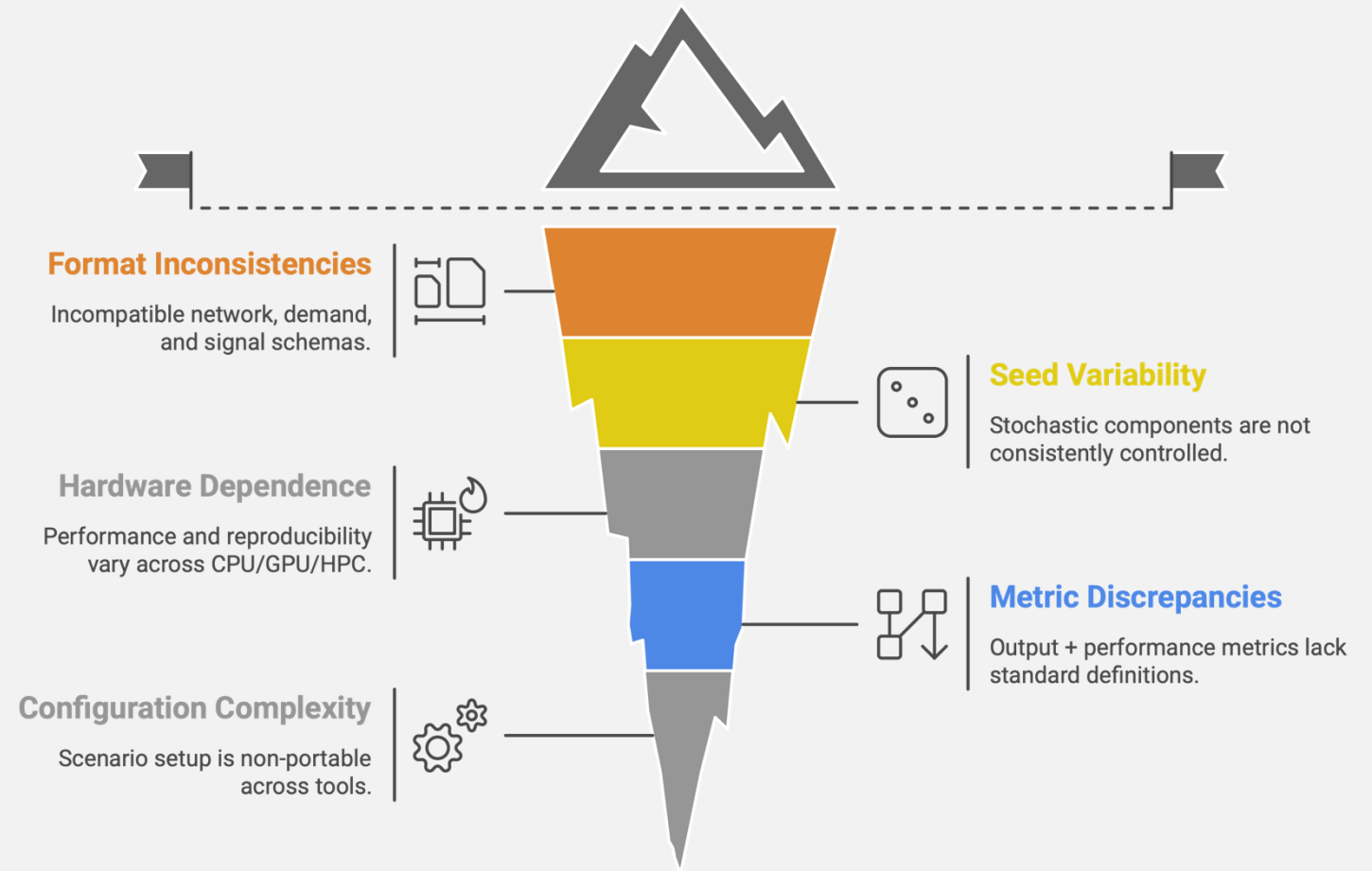
- Cities rely on simulation to test policies, signals, and infrastructure when real-world trials are expensive or risky.
- Today's simulator studies are hard to compare because inputs, assumptions, and configs differ across tools.
- Without standardized scenarios and controlled execution, output differences may reflect software quirks, not traffic behavior.
- The field lacks an open, reproducible cross-simulator benchmark that spans CPU, GPU, and HPC settings.

RESEARCH GAP

The field still lacks a standardized foundation for fair, repeatable evaluation.

SIMFORGE targets these gaps with a canonical bundle, translation layer, and reproducible execution and evaluation.

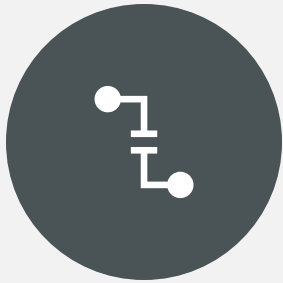
The Cross-Simulator Reproducibility Gap





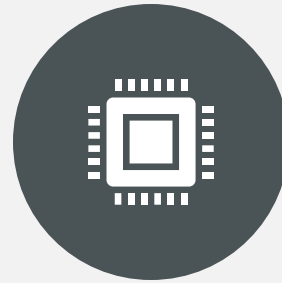
RESEARCH QUESTIONS & EXPECTED CONTRIBUTIONS

RESEARCH QUESTIONS



RQ1: Fidelity

Under identical canonical inputs, how closely do simulators reproduce observed traffic counts, link-level flows, and trip-time patterns across cities and demand levels?



RQ2: Scalability

How does runtime performance and throughput scale with agent count across CPU, GPU, and HPC environments?



RQ3: Reproducibility

With fixed seeds, deterministic adapters, and containerized environments, what run-to-run variance remains, and which components contribute most?



RQ4: Trade-offs

What fidelity throughput trade-offs emerge across simulators, and are they consistent across urban topologies, congestion regimes, and compute architectures?

EXPECTED CONTRIBUTIONS

C1: Canonical Schema & Validators

A simulator-agnostic scenario bundle with validation and pre-run integrity checks: network.xml, demand.csv, signals.xml, config.xml and manifest.

C2: Deterministic Adapters

Auditable one-to-one mappings from the canonical bundle to SUMO, MATSim, POLARIS, LPSim, QarSUMO, including: parameter translation, unit normalization, and time-step alignment.

C3: Reproducible Execution Pipeline

Version-pinned, containerized workflows (OCI/Singularity) with: fixed seeds, consistent time-step policies, and hash-verified inputs/outputs across CPU/GPU/HPC.

C4: Hardware-Normalized Evaluation

Unified KPIs with confidence intervals:

Fidelity: RMSE, GEH, KS

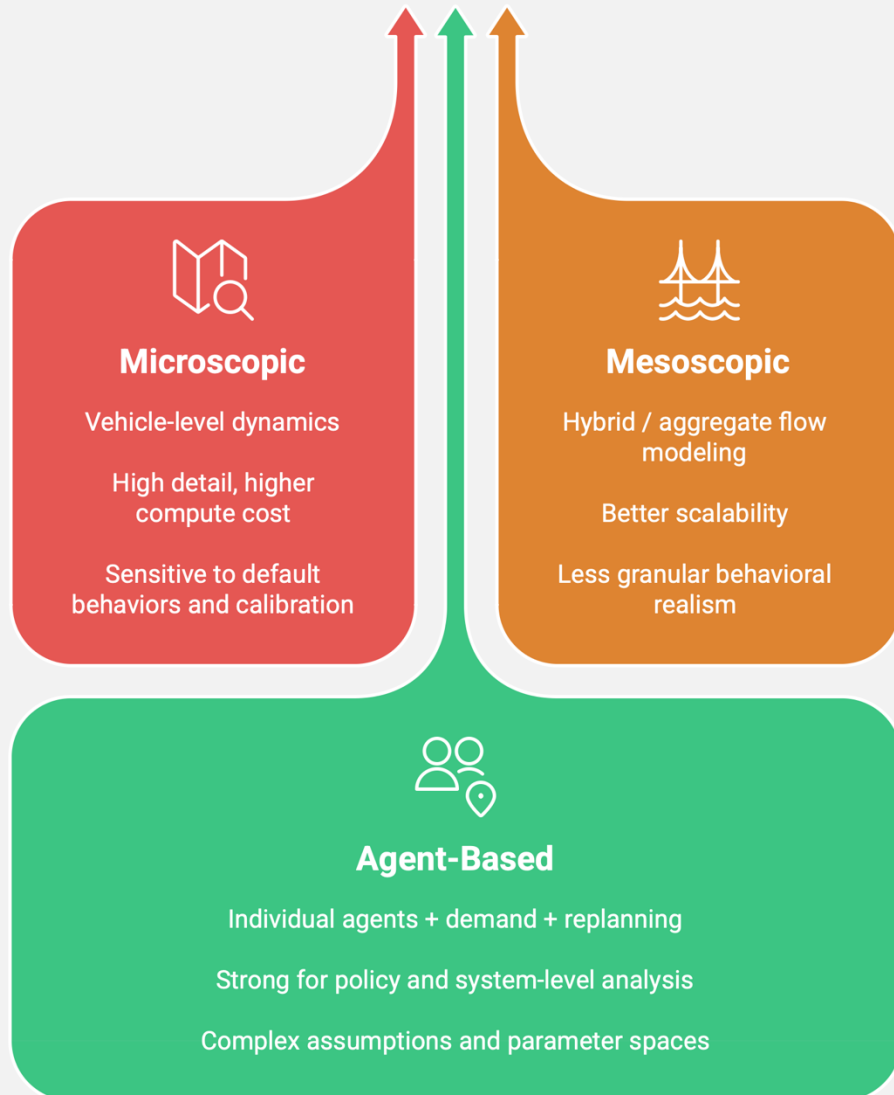
Scalability: runtime vs agents, vehicles/sec per core, vehicles/sec per watt

Reproducibility: $R = 1 - \sigma/\mu$



BACKGROUND & SIMULATOR SCOPE

Approaches to Traffic Modeling



WHY THIS MATTERS FOR SIMFORGE

Even before implementation differences, these abstractions create:

- Different input expectations
- Different default assumptions
- Different output interpretations
- Different scaling profiles across CPU/GPU/HPC

SIMFORGE is needed because “same city, same policy” does not automatically mean the same experiment across simulator families.

TARGET SIMULATORS (PRIMARY + BACKUP)

Simulator	Modeling Family	Role	Why Selected
SUMO	Microscopic	Primary	Open-source baseline for canonical input and controlled CPU execution.
MATSim	Agent-Based	Primary	Represents a distinct modeling family for system-level policy and demand effects.
QarSUMO	Microscopic (Parallel Variant)	Primary	Scalability stress-test under standardized scenarios, aligned with multi-core/HPC evaluation.
POLARIS	Agent-Based	Backup	Expands family coverage if integration and access are reliable.
LPSim	Large-Scale / Performance-Oriented	Backup	Additional high-scale comparator if reproducible execution is feasible.



SIMFORGE FRAMEWORK & WORKFLOW

SIMFORGE: PROPOSED FRAMEWORK

What SIMFORGE provides

- Canonical scenario bundle (network, demand, signals, metadata)
- Translation layer for multi-simulator input generation
- Reproducible execution protocol (pinned env and controlled seeds)
- Unified evaluation suite (outputs and performance)

What SIMFORGE enables

- Fair cross-engine comparisons
- Benchmarking across CPU/GPU/HPC

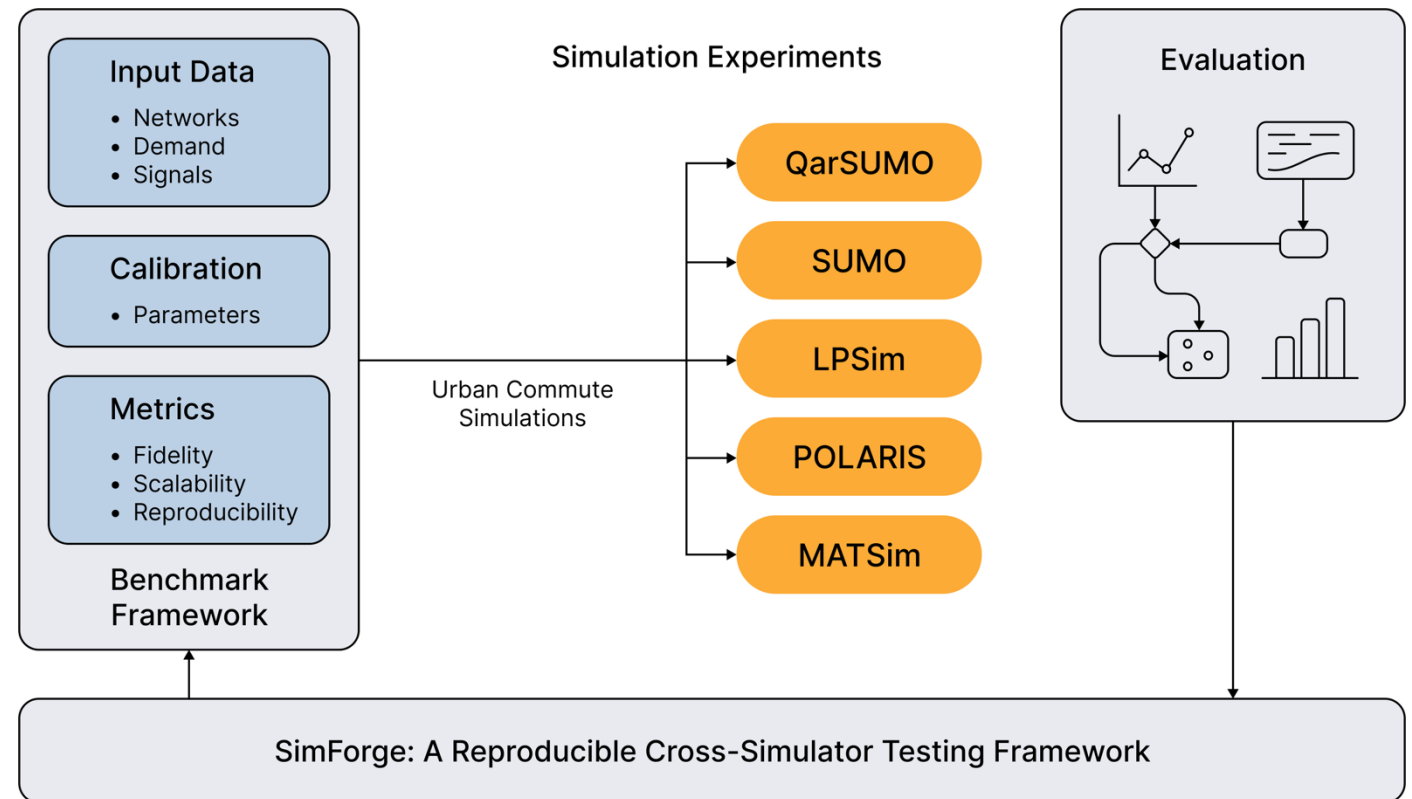
HOW SIMFORGE WORKS

Canonical Scenario Bundle

Parameter Alignment and Translation

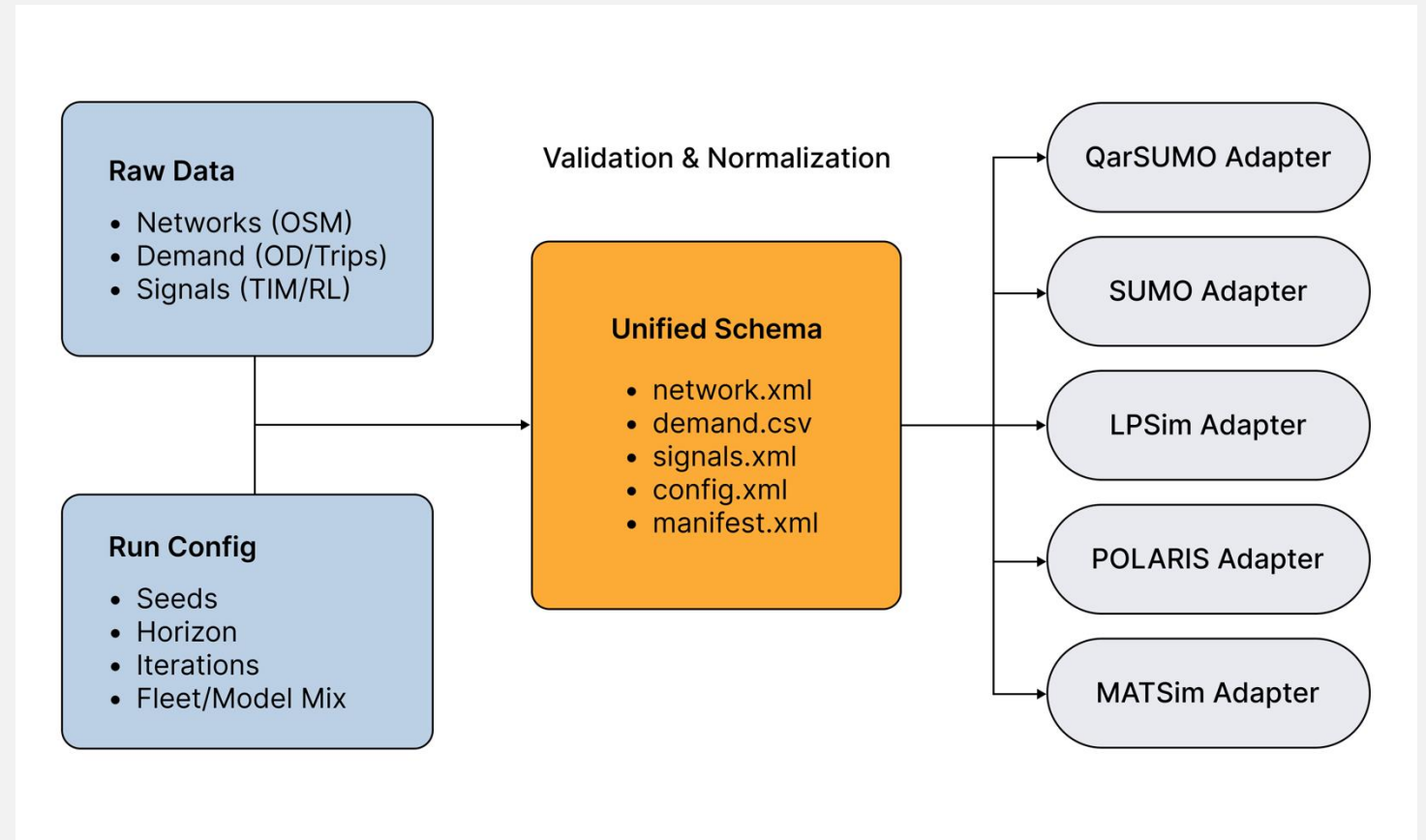
Parallel Simulator Runs

Unified Evaluation



CANONICAL BUNDLE & DETERMINISTIC ADAPTERS

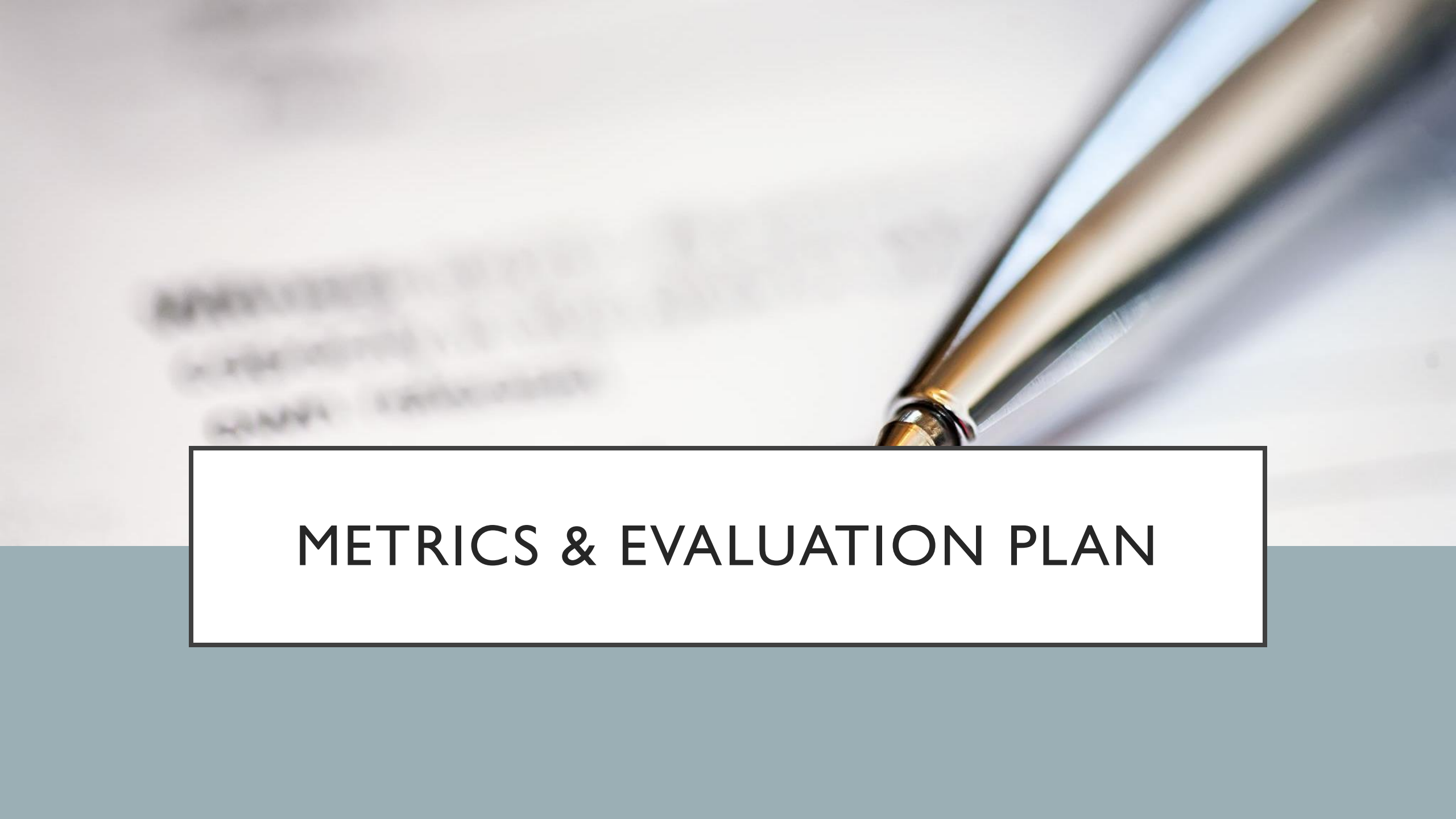
- Canonical/Unified Schema defines the experiment once with standardized network, demand, signals, and run metadata.
- Validation and normalization enforce integrity before execution (schema, cross-file checks).
- Deterministic adapters convert the canonical bundle into each simulator's native inputs with auditable parameter and unit alignment.
- Calibration is intentionally minimal and standardized to preserve fairness across engines.



VALIDATION PIPELINE (PRE-RUN INTEGRITY CHECKS)

- ID consistency across network, demand, and signals
- CRS declared and consistent
- Unit normalization (meters, seconds, km/h)
- Referential integrity (demand references valid nodes/links)
- Signal intersection alignment
- Manifest checksums verify input integrity

This prevents silent errors and makes the benchmark reproducible by construction.



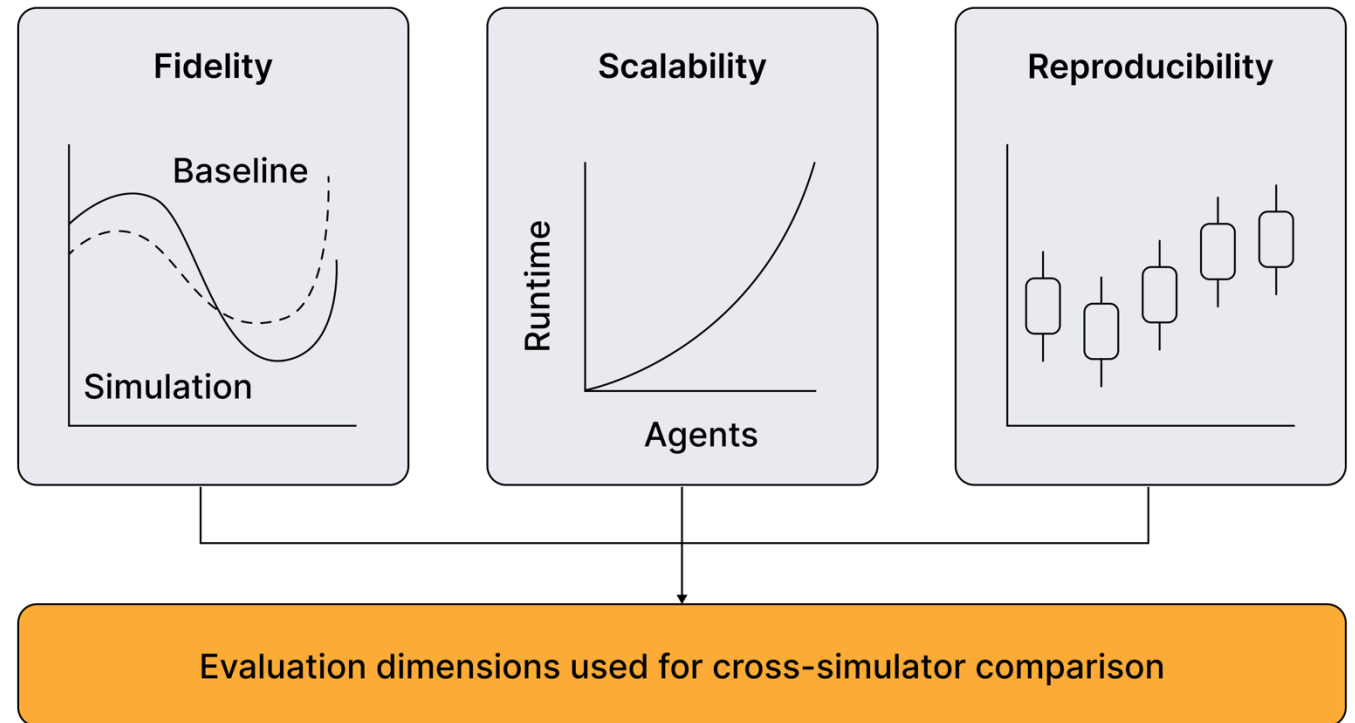
METRICS & EVALUATION PLAN

METRICS MAPPED TO RESEARCH QUESTIONS

RQ	Dimension	Primary Measures	Notes
RQ1	Fidelity	RMSE, GEH, KS: counts, link flows, trip/travel times	Compare against observed data across cities and demand levels
RQ2	Scalability	Runtime vs agents: vehicles/sec (or trips/sec), scaling efficiency	Report per core and per watt where available
RQ3	Reproducibility	Run-to-run variance, seed sensitivity: $R = 1 - \sigma/\mu$	Isolate variance sources (routing, scheduling, I/O)
RQ4	Trade-offs	Fidelity vs throughput	Check stability across topology, congestion and architecture

EVALUATION DIMENSIONS

- Evaluates Fidelity, Scalability, Reproducibility, plus Trade-offs across engines.
- All comparisons use identical canonical scenarios and deterministic adapters.
- Executions are controlled via pinned containers and fixed seeds where applicable.
- Results reported with hardware normalization and confidence intervals.



This work additionally analyzes stability, runtime, and efficiency as operational lenses across environments.

EXPERIMENTAL DESIGN (RUN MATRIX)

Dimension	What's Fixed	What Varies	Purpose
Scenario	Canonical bundle & validators	City/network & demand tier	Ensures identical inputs
Simulators	Primary set	Backup set if needed	Cross-engine comparison
Environments	Pinned containers	CPU / GPU / HPC	Core thesis axis
Calibration	Minimal standardized rules	Small engine-specific parameter subset	Prevents unfair tuning
Execution	Fixed seeds (where applicable)	Repeats per setting	Quantify stability
Evaluation	Unified KPIs & normalization	Per dimension & trade-off analysis	Comparable results

Total runs scale as: (#cities × #demand tiers × #simulators × #environments × #repeats)

Example: 3 cities × 3 demand tiers × 3 simulators × 3 environments × 10 repeats = 810 runs

BENCHMARK SCENARIOS & DATA SOURCES

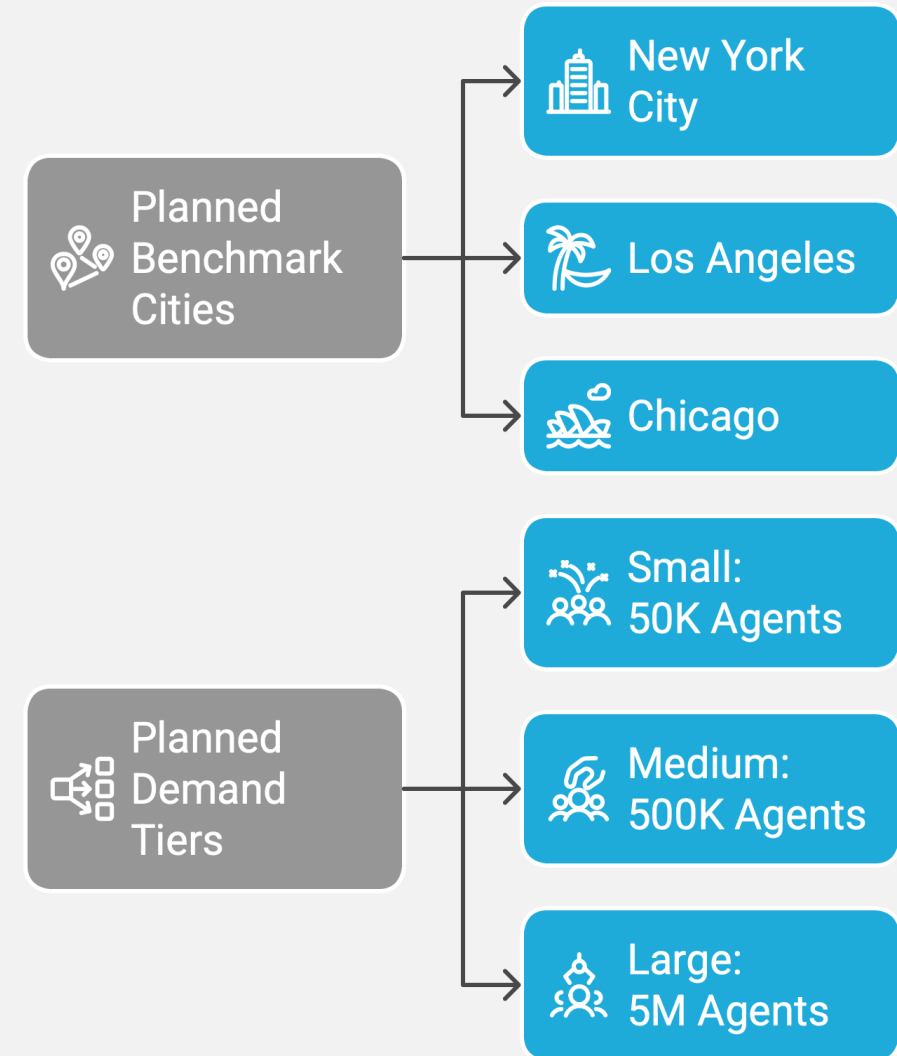
What Each Scenario Includes (Canonical Bundle)

- Network (OSM-derived, validated topology)
- Demand (tiered scaling)
- Signals (where available)
- Config and Manifest (seeds, policies, provenance)

Data Sources

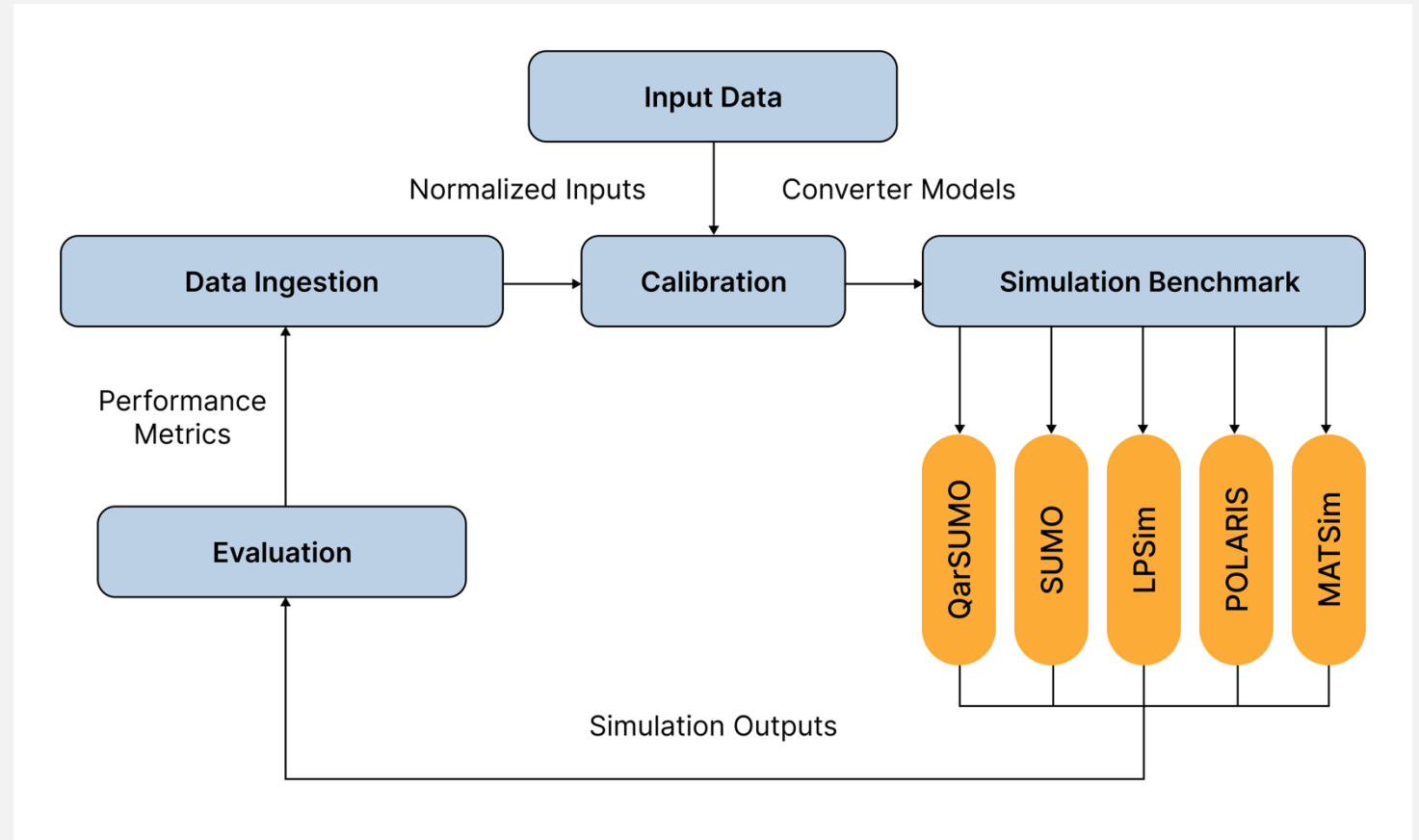
- Networks: extracted from OpenStreetMap (OSM)
- Demand: derived from public datasets / OD sources OR synthesized with fixed seeds
- Signals: public DOT releases when available or standardized defaults

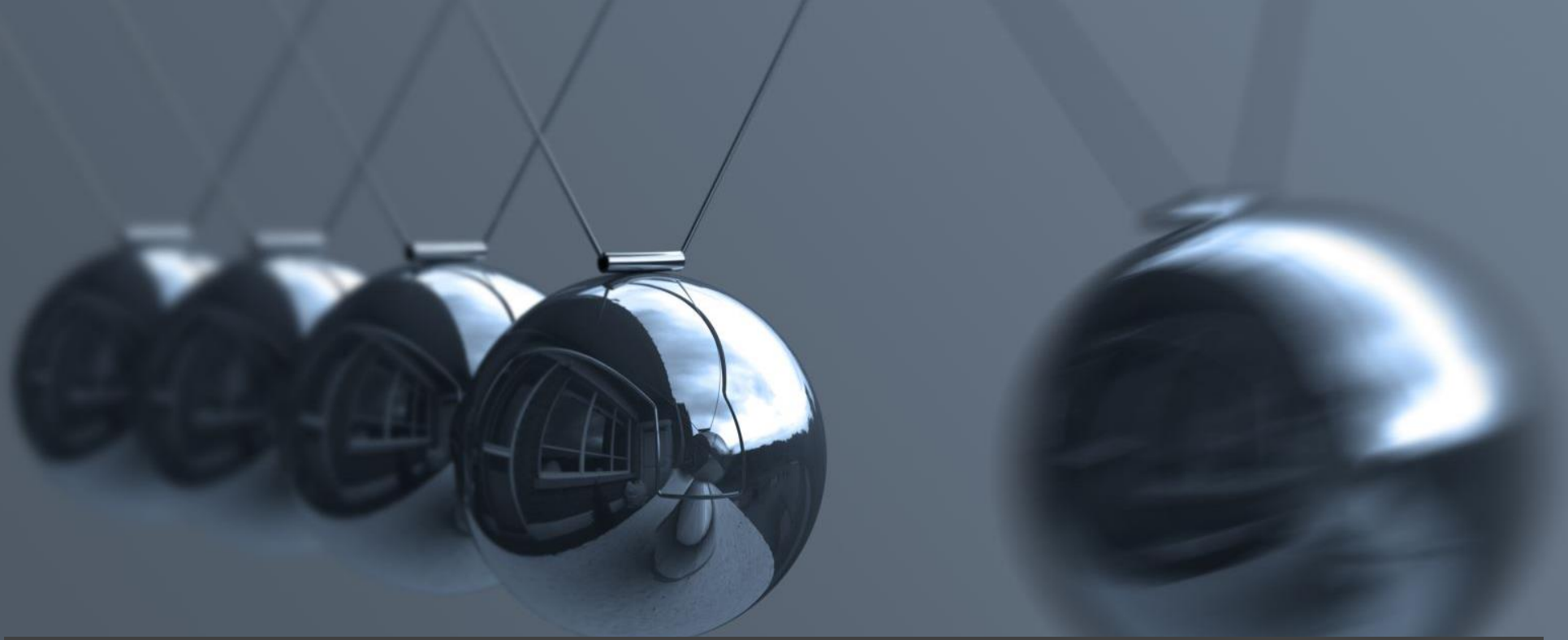
Benchmark Scenario Design



BENCHMARK PROCESS (END-TO-END)

- Canonical inputs are normalized and validated first (schema rules, cross-file integrity and manifest hashing).
- The workflow then runs a controlled, minimal calibration phase followed by a strict evaluation phase, no post tuning allowed.
- Deterministic adapters translate the validated bundle into engine-native inputs.
- Outputs are aggregated into a unified evaluation layer for fidelity, scalability, reproducibility, and trade-off analysis.





RISKS & TIMELINE

RISKS & MITIGATION

RISK 1: ADAPTER AMBIGUITY

Problem:

Canonical native mappings may not be perfectly equivalent across engines.

Mitigation:

- Golden “micro-scenarios” for sanity checks
- Unit/time-step normalization rules
- Adapter audit logs and diffable outputs

RISK 2: GPU/HPC NONDETERMINISM

Problem:

Parallel execution may introduce run-to-run drift.

Mitigation:

- Fixed seeds (where applicable)
- Controlled time-step policies
- Repeat runs and stability reporting

RISKS & MITIGATION

RISK 3: DATA GAPS (SIGNALS/DEMAND)

Problem:

Public signal plans may be incomplete:
demand uncertainty varies by city.

Mitigation:

- Standardized fallback signal templates
- Deterministic demand synthesis
- Manifest-level provenance tracking

RISK 4: RUNTIME AND QUEUE DELAYS

Problem:

HPC scheduling could slow the full matrix.

Mitigation:

- Staged execution (city-by-city)
- CPU baselines first
- Prioritized subsets to ensure publishable results even if scope shrinks

TIMELINE

Finalize canonical schema &
validators

Dec 2025 – Jan 2026

Containerized execution &
CPU baseline runs

Apr 2026

Writing, revisions, and defense
prep

Jun – Jul 2026

Feb – Mar 2026

Build deterministic adapters
for primary simulators

May 2026

GPU/HPC scaling & full
evaluation & trade-off analysis



THANK YOU!